ForecastLabAI — RAG pipeline with pgvector

Problem

Show that the project has a real RAG subsystem (not just an LLM call) and defend its choices.

Instant solution

Indexing: Markdown / OpenAPI docs → chunking → embeddings → PostgreSQL + pgvector. **Retrieval:** question → query embedding → cosine search (HNSW) → top-k chunks → RAG agent → answer with citations.

Piece	Implementation
Chunking	`MarkdownChunker` splits on headings; `OpenAPIChunker` one chunk per endpoint (`rag/chunkers.py`)
Embeddings	OpenAI `text-embedding-3-small` (default) or Ollama `nomic-embed-text` (`rag/embeddings.py`)
Store	pgvector, HNSW index, `vector_cosine_ops` (`rag/models.py`)
Defaults	chunk 512 tokens · overlap 50 · top-k 5 · similarity threshold 0.7
Re-index	SHA-256 content hash → unchanged docs are skipped (idempotent)
Answer	`RAGAnswer`: answer · confidence (low/medium/high) · sources · no_evidence

"Why pgvector?"

> *The app already runs on PostgreSQL for business data and model metadata. pgvector keeps vector search in the same database: similarity search and HNSW indexing are enough for this corpus, with one less system to operate. At larger scale I'd benchmark a dedicated vector store before adding it.*

"How do you reduce hallucinations?"

> *Answers are grounded in retrieved chunks; the agent checks evidence and returns a structured answer with citations, or says evidence is insufficient. I'd measure retrieval recall, citation correctness and faithfulness.*

Why it works / gotchas

- Defaults are defaults, not tuned optima.
- Grounding reduces hallucinations; it does not guarantee zero.
- RAG ≠ fine-tuning: RAG adds knowledge at inference time, fine-tuning changes weights. They can be combined.

Diagram brief

- Type: two-lane flowchart.
- Lane "Indexing": Docs (Markdown · OpenAPI) → Chunker (heading / endpoint) → Embedding provider (OpenAI · Ollama) → pgvector table (chunks · vectors · SHA-256 hash · HNSW).
- Lane "Retrieval": User question → Query embedding → Cosine top-k ≥ 0.7 → RAG Assistant (PydanticAI) → RAGAnswer (answer · confidence · sources · no_evidence).
- Shared node in the middle: PostgreSQL + pgvector.